DEPARTMENT OF SOFTWARE ENGINEERING

Web Engineering

SPRING-2026

SE6322

BSSE – 6 Blue

Project Report

Team Stream

Submitted by:

- **Mutahara Batool (008)**
- **Umais Javed (042)**

Submitted to:

- **Sir Museb Khalid**

16th July,2026

Table of Contents

Team Stream

Introduction

Team Stream is a real-time, full-stack chat application developed as a semester project for the Web Engineering course at the University of Sialkot. The system combines a Vue 3 single-page frontend with a Node.js/Express backend, MongoDB persistence, and Socket.io for live communication.

Team Stream allows users to register and authenticate, join themed chat rooms, exchange messages in real time, send private direct messages, and share files, all within a responsive single-page application. This report presents the project as one cohesive system rather than as separate frontend and backend submissions, tracing how the two layers integrate to deliver the full user experience.

Project Overview

The application supports account registration and login, room-based group chat, live typing indicators, join/leave notifications, private one-to-one messaging, file sharing, message history, and persistent sessions. The frontend handles presentation and user interaction, while the backend manages authentication, data persistence, and real-time event distribution between connected clients.

Technology Stack

Layer	Technology	Purpose
Frontend	Vue 3 (Composition API)	UI framework, component logic, reactivity
Frontend	Vite	Development server, build tool, hot reload
Frontend	Vue Router 4	SPA navigation and protected routing
Frontend	Pinia	Global state management
Frontend	Axios	HTTP requests to the REST API

Layer	Technology	Purpose
Frontend	Socket.io Client	Real-time WebSocket communication in the browser
Backend	Node.js / Express.js	REST API server and application logic
Backend	MongoDB / Mongoose	Data persistence and schema modelling
Backend	Socket.io	Real-time event broadcasting between clients
Backend	JWT	Stateless authentication tokens
Backend	bcrypt	Password hashing
Backend	Multer	File upload handling

System Architecture

Team Stream follows a client-server architecture. The Vue 3 SPA runs entirely in the browser and communicates with the backend over two channels: Axios-based REST calls for authentication and standard CRUD operations, and a persistent Socket.io connection for real-time events such as messages, typing indicators, and presence updates. The Express server validates requests, applies JWT-based authentication middleware, persists data to MongoDB, and re-broadcasts real-time events to relevant connected clients.

Frontend Implementation

The frontend is built as a Single Page Application using Vue 3 and Vite. It is organized into two directories under client/src/: views/ for full-page route components and components/ for smaller, reusable UI pieces, following the single-responsibility principle.

Client-Side Features

- Login and Sign Up screens with input validation and inline error display
- Toggle between Login and Sign Up mode on the same page
- Room selection screen shown after successful authentication

- Real-time chat interface with message bubbles (own messages on the right, others on the left)
- Online users list displayed in a sidebar
- Join and leave notifications shown in the center of the chat
- Private direct messaging panel opened by clicking a user
- Typing indicator showing “username is typing…” in real time
- Message history loaded on joining a room
- Protected routes that redirect unauthenticated users to the login page
- Persistent login using localStorage so sessions survive a page refresh

SPA Routing

Vue Router 4 handles all client-side navigation using **createWebHistory()** for clean URLs. A global navigation guard **(router.beforeEach)** protects every route except the login page, redirecting unauthenticated users to /login regardless of the URL requested.

Path	Route Name	Component	Access
/login	login	AuthView.vue	Public
/rooms	rooms	HomeView.vue	Protected
/chat	chat	ChatView.vue	Protected

App.vue is the root component and contains a single <RouterView /> tag, which Vue Router uses to inject the appropriate page component based on the current URL without triggering full page reloads.

Modular Component Architecture

Component	Type	Responsibility
App.vue	Root	Wraps the entire app, renders RouterView
AuthView.vue	View	Login/Signup form, calls the API via Axios, stores the token in Pinia
HomeView.vue	View	Room selector dropdown, welcome message, logout

Component	Type	Responsibility
ChatView.vue	View	Parent of all chat components, manages the real-time connection and typing state
MessageList.vue	Component	Renders the message array, differentiates own vs. others' messages, auto-scrolls
MessageInput.vue	Component	Text input, sends on Enter or button click, emits typing events
UserList.vue	Component	Lists online users; clicking a user opens their DM panel
PrivateChat.vue	Component	Floating DM panel showing private message history

Inter-Component Communication

Data flows downward via **props** e.g., messages and currentUser passed into MessageList, socket and room passed into MessageInput and events flow upward via $emit e.g., MessageInput **emits** a “send” event that ChatView handles. This keeps a clean, unidirectional data flow throughout the component tree.

Global State Management with Pinia

Two Pinia stores manage application-wide state so components can access shared data without prop-drilling. The auth store holds the JWT token and username in localStorage and exposes isLoggedIn(), which the router guard uses to protect routes. The chat store manages real-time chat data, clearing messages on every room switch to prevent stale data, and tracks private DM state per conversation.

Backend Implementation

The backend is built with Node.js and Express.js, exposing REST endpoints for authentication and core resources while using Socket.io for real-time event handling. It is responsible for validating and persisting data, enforcing authentication and authorization, and relaying real-time events to the appropriate connected clients.

REST API

The API exposes endpoints covering authentication, room management, user profiles, file uploads, and message search. Authentication endpoints issue and validate JWTs, while protected endpoints require a valid token before granting access to room, profile, or messaging resources.

Database Design

Data is persisted in MongoDB via Mongoose across three primary collections:

- **Users:** account credentials and profile information
- **Messages:** room and private chat message records, including history
- **Rooms:** chat room definitions available for users to join

Security

- Passwords hashed with bcrypt before storage
- JWT-based authentication for stateless session verification
- Protected API routes that reject requests without a valid token
- Sensitive configuration values (database URIs, JWT secrets) kept in environment variables

Real-Time Communication

Socket.io powers all live functionality: broadcasting messages to everyone in a room, relaying typing indicators, delivering private direct messages between two users, and loading message history when a user joins a room or opens a DM thread.

File Uploads

Multer handles multipart file uploads on the server, enabling users to share files within chat conversations alongside text messages.

Frontend-Backend Integration

The Vue frontend communicates with the backend over two coordinated channels. Axios is used for stateless REST operations such as login, signup, and profile actions, with the JWT attached to authenticated requests. Socket.io maintains a persistent connection for everything that must happen instantly: new messages, typing status, presence (online

users), and private messages. Together, these channels let the UI stay reactive while the server remains the single source of truth for persisted data.

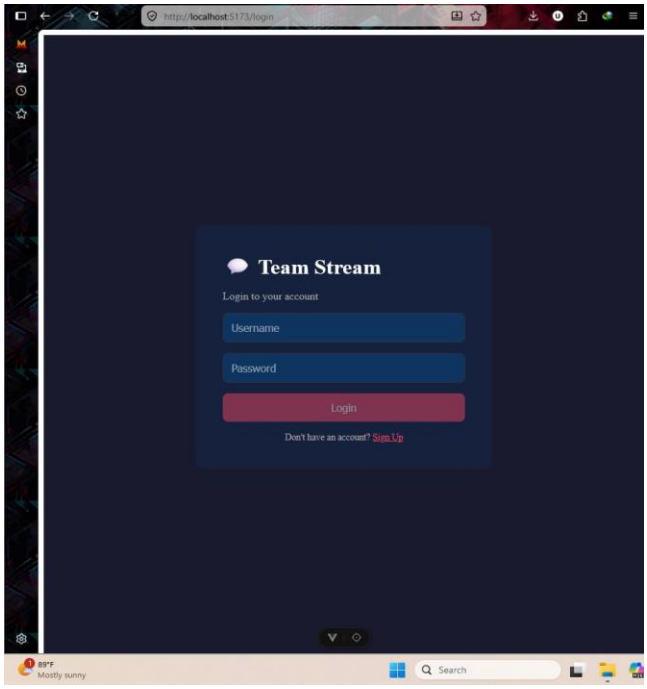
Key Features Summary

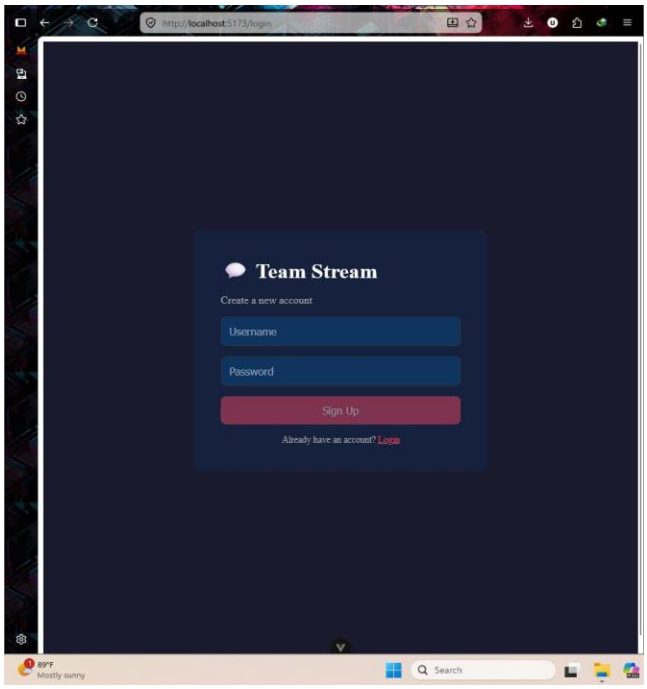
- User authentication (signup, login, persistent sessions)
- Multiple themed chat rooms with real-time group messaging
- Typing indicators and join/leave notifications
- Private one-to-one direct messaging
- File sharing within conversations
- Message history and search
- Protected routes and profile management

Website Screenshots

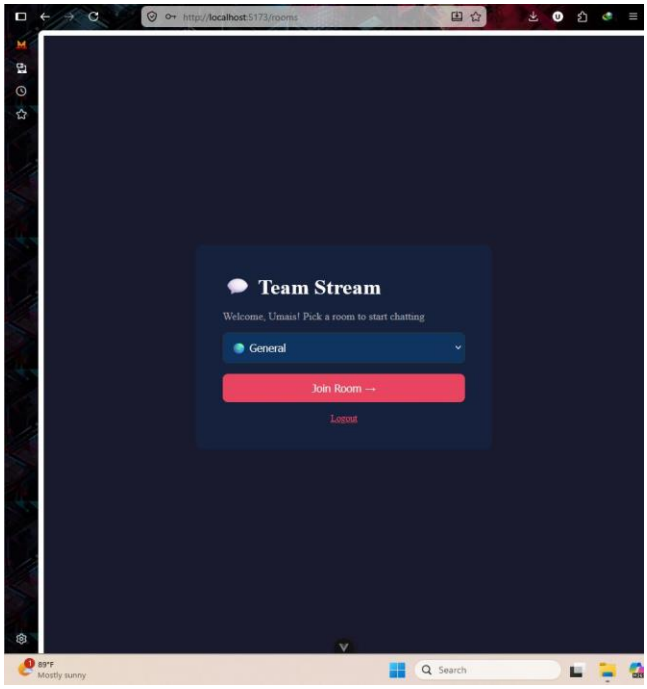
The following screenshots document the running application at http://localhost:5173, demonstrating the integrated frontend and backend in action.

Login and Sign Up

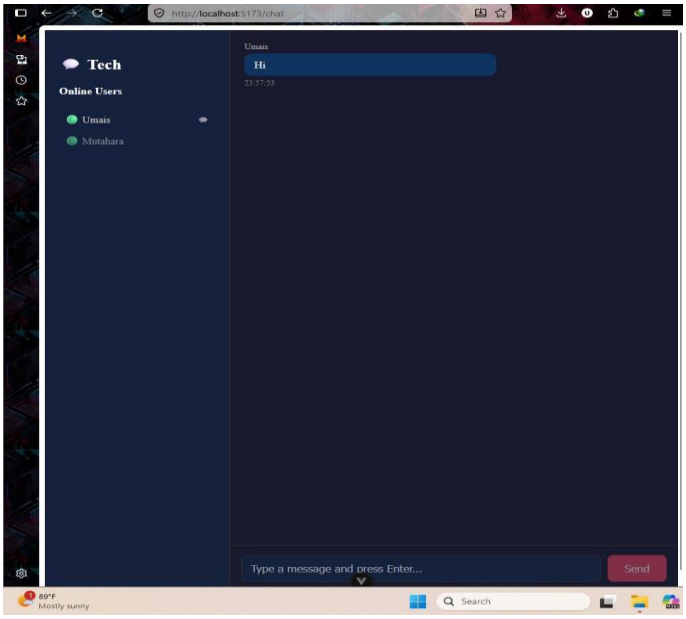




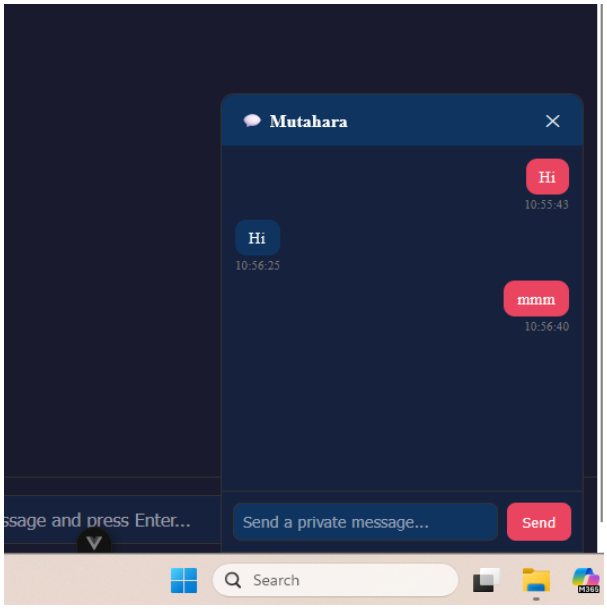
Room Selection



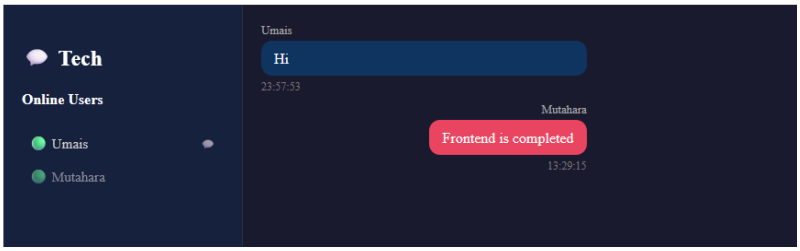
Main Chat Interface



Private Direct Messaging



Two Users Chatting Simultaneously



Challenges

- Managing room switching without leaking stale messages between rooms
- Configuring CORS between the Vite dev server and the Express API
- Correctly configuring Socket.io namespaces/rooms for group vs. private messaging
- Integrating MongoDB schemas with real-time data flows
- General debugging across the client-server real-time pipeline

Conclusion

Team Stream successfully demonstrates a complete, modern full-stack web application. The Vue 3 frontend delivers a responsive, protected, component-driven SPA, while the Express/MongoDB backend provides secure authentication, persistent storage, and a REST API. Socket.io ties the two together for real-time messaging, typing indicators, and private chat. Together, these layers fulfill all functional requirements of the assignment and demonstrate the integration of modern frontend and backend engineering practices in a single, cohesive system.